

Informática Avanzada

Introducción

Introducción

Los objetivos del curso

- Obtener conocimiento de **principios de programación**
- Aplicar el paradigma de **programación orientada a objetos**
- Implementar pequeños **sistemas de software y aplicaciones**
- **Evaluar la calidad** de un sistema de software
- Obtener conocimientos de la naturaleza y características principales de **sistemas operativos**.



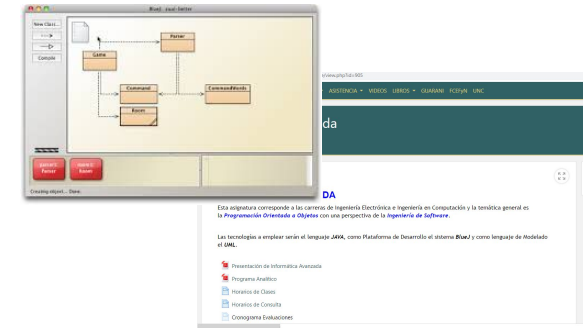
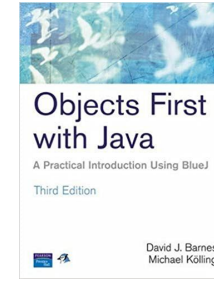
Recursos

- **Programación Orientada a Objetos con Java** - *David J. Barnes & Michael Kölling* - Pearson (2003-2017) (6ta edición)
- **Sistemas operativos: aspectos internos y principios de diseño** - William Stallings - Pearson (2011) (5ta edición)

-  BlueJ - <https://www.bluej.org/>

-  Java 11 (JDK) - <https://www.oracle.com/java/technologies/java-se-glance.html>

-  LEV - Informática Avanzada <https://fcefyn.aulavirtual.unc.edu.ar/course/view.php?id=2900>

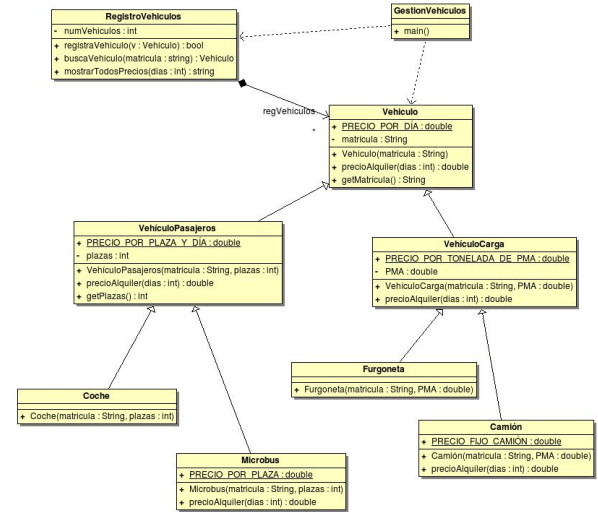


Programación Orientada a Objetos

- Objetos y Clases
- Definición de Clases
- Interacción entre Objetos
- Agrupación de Objetos
- Bibliotecas
- Testing, Mantenimiento y Debugging
- Diseño de Clases
- Herencia y Polimorfismo
- Estructuras flexibles y extensibles
- Manejo de Errores

Sistemas Operativos

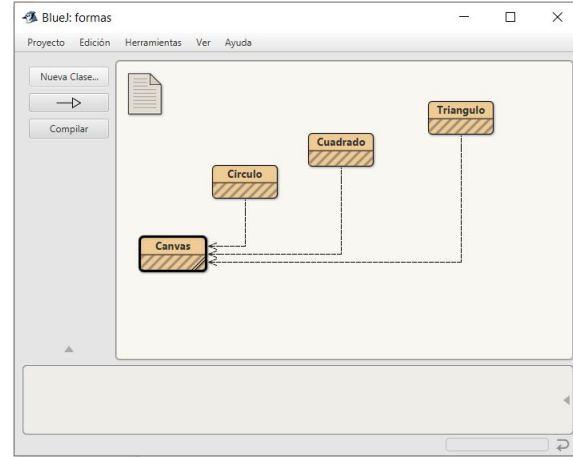
- Funciones del Sistema Operativo.
- Interfaces con el usuario
- Interfaces con las aplicaciones
- El núcleo del Sistema Operativo



Creando objetos en BlueJ

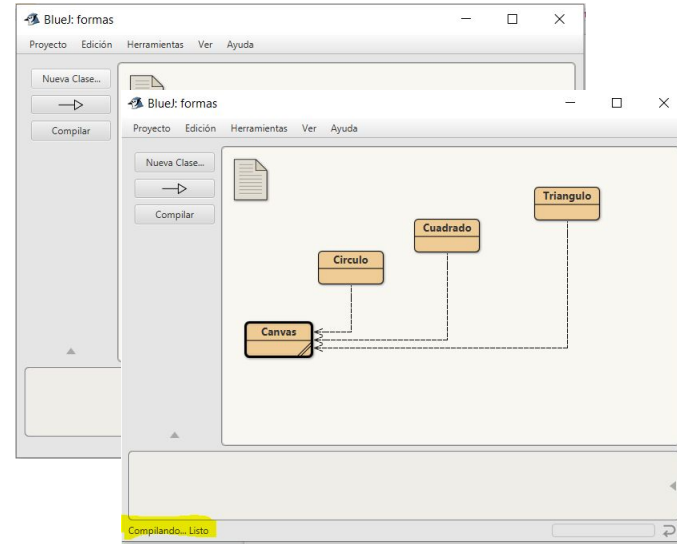
BlueJ - Proyecto Formas

- Abrir el proyecto “Formas”
 - Diagrama de Clases
- Compilar
 - JAVA es un lenguaje que requiere complicación y luego es interpretado
- Instanciar un Objeto
 - Identificamos cada instancia para poder referenciarla
- Pueden crearse uno o muchos objetos de una Clase
 - Todos son independientes



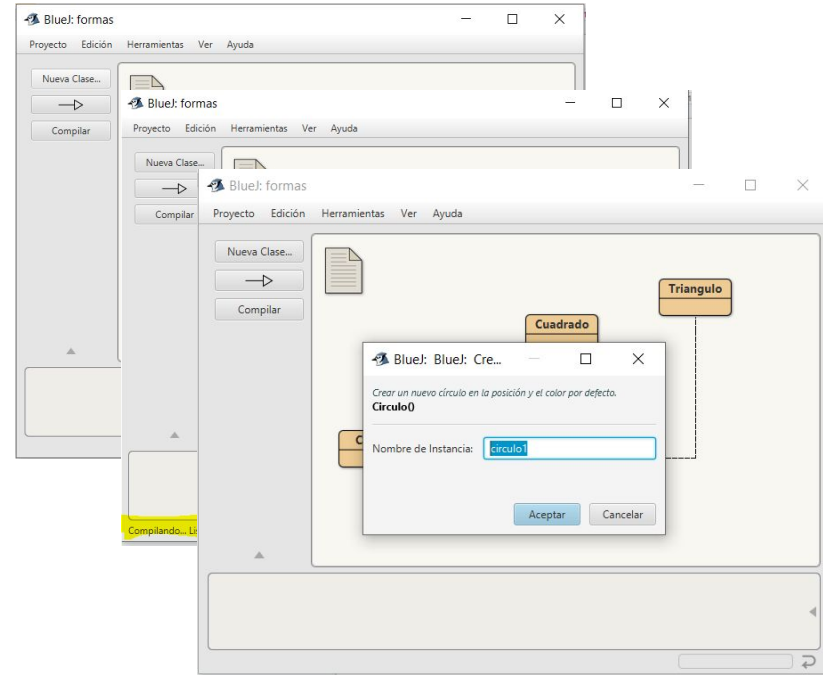
BlueJ - Proyecto Formas

- Abrir el proyecto “Formas”
 - Diagrama de Clases
- Compilar
 - JAVA es un lenguaje que requiere complicación y luego es interpretado
- Instanciar un Objeto
 - Identificamos cada instancia para poder referenciarla
- Pueden crearse uno o muchos objetos de una Clase
 - Todos son independientes



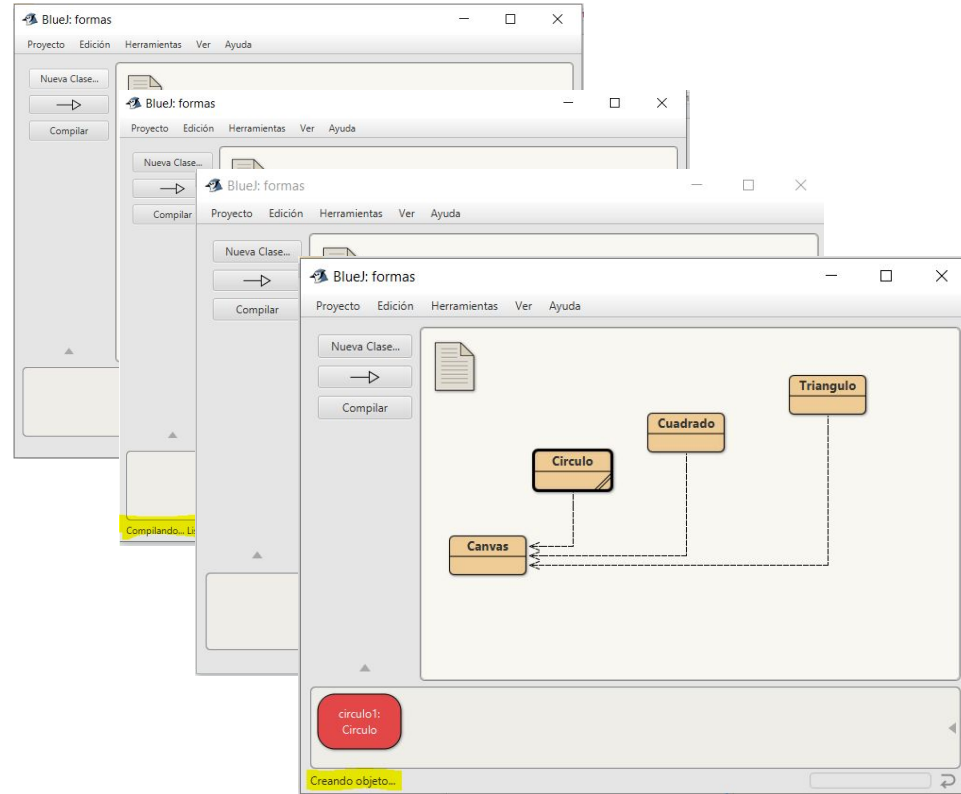
BlueJ - Proyecto Formas

- Abrir el proyecto “Formas”
 - Diagrama de Clases
- Compilar
 - JAVA es un lenguaje que requiere complicación y luego es interpretado
- Instanciar un Objeto
 - Identificamos cada instancia para poder referenciarla
- Pueden crearse uno o muchos objetos de una Clase
 - Todos son independientes



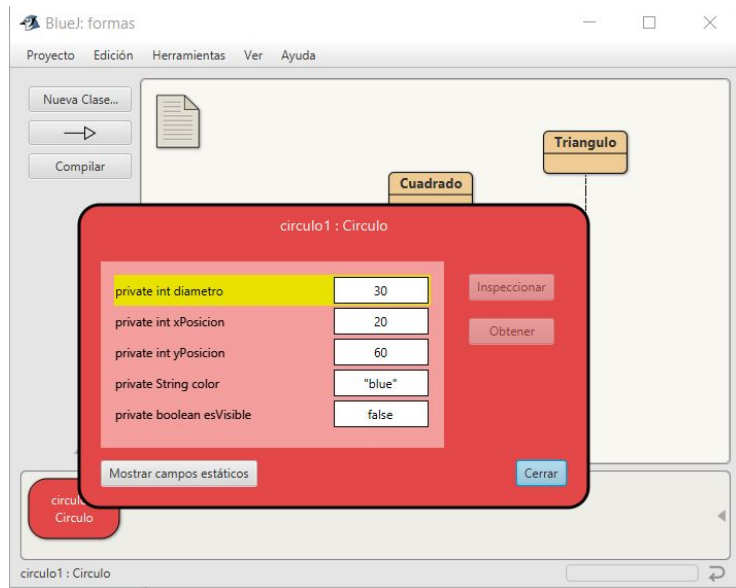
BlueJ - Proyecto Formas

- Abrir el proyecto “Formas”
 - Diagrama de Clases
- Compilar
 - JAVA es un lenguaje que requiere complicación y luego es interpretado
- Instanciar un Objeto
 - Identificamos cada instancia para poder referenciarla
- Pueden crearse uno o muchos objetos de una Clase
 - Todos son independientes



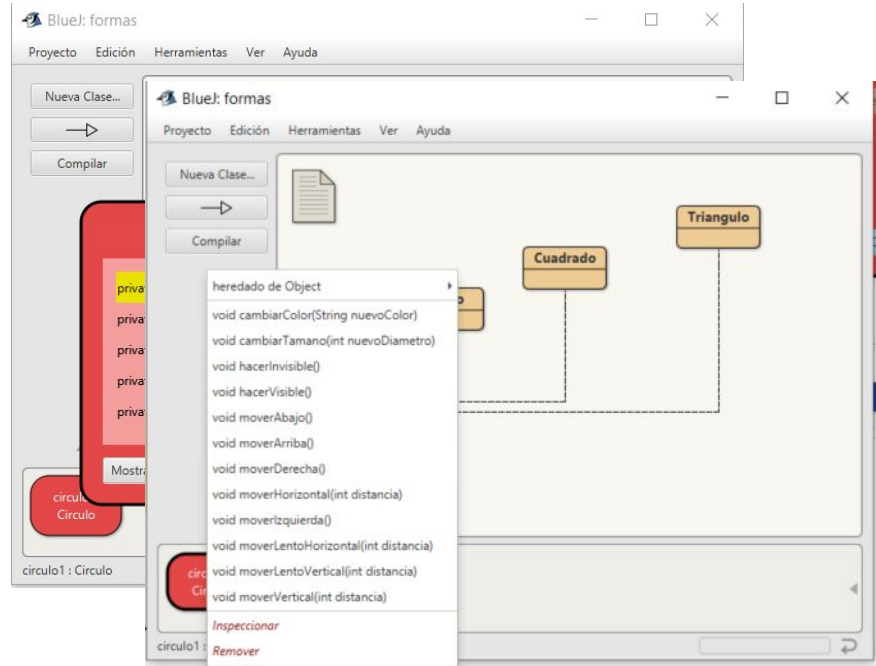
BlueJ - Proyecto Formas (cont.)

- Examinar un objeto y su estado
 - Los objetos poseen atributos, definidos por la clase
- Invocar acciones a un objeto
 - Los objetos pueden realizar acciones, a través de sus **métodos**
 - Pueden requerir información adicional para ejecutarse, a través de **parámetros**
 - Y pueden **retornar valores** a quien lo invoca
- Crear múltiples instancias de una clase
 - Cada objeto tiene su propio conjunto de valores para los atributos



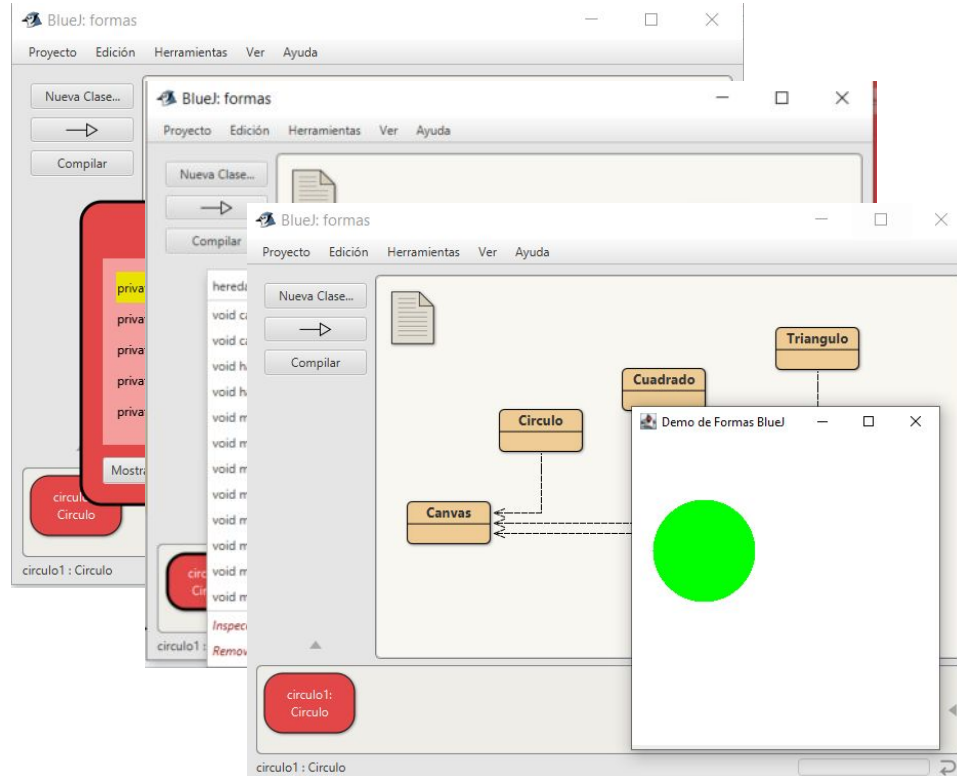
BlueJ - Proyecto Formas (cont.)

- Examinar un objeto y su estado
 - Los objetos poseen atributos, definidos por la clase
- Invocar acciones a un objeto
 - Los objetos pueden realizar acciones, a través de sus **métodos**
 - Pueden requerir información adicional para ejecutarse, a través de **parámetros**
 - Y pueden **retornar valores** a quien lo invoca
- Crear múltiples instancias de una clase
 - Cada objeto tiene su propio conjunto de valores para los atributos



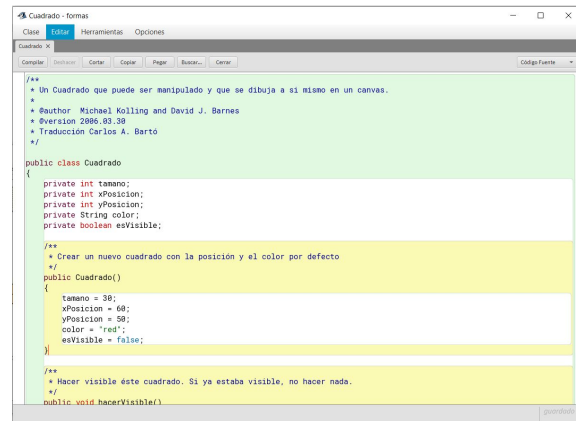
BlueJ - Proyecto Formas (cont.)

- Examinar un objeto y su estado
 - Los objetos poseen atributos, definidos por la clase
- Invocar acciones a un objeto
 - Los objetos pueden realizar acciones, a través de sus **métodos**
 - Pueden requerir información adicional para ejecutarse, a través de **parámetros**
 - Y pueden **retornar valores** a quien lo invoca
- Crear múltiples instancias de una clase
 - Cada objeto tiene su propio conjunto de valores para los atributos



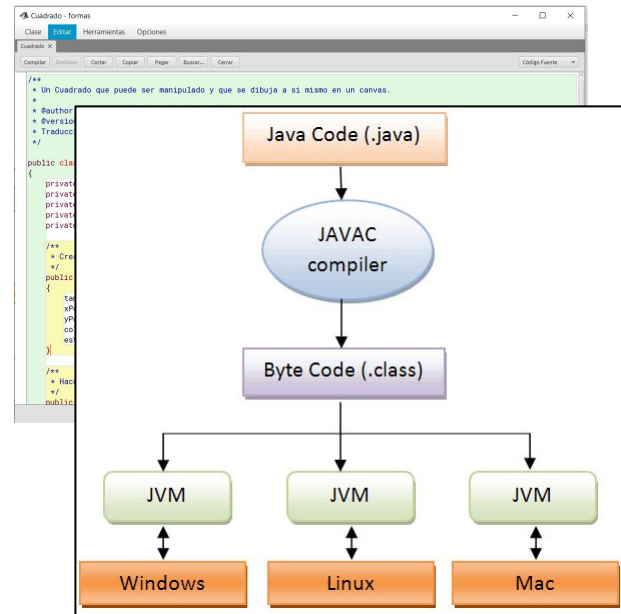
BlueJ - Código Fuente

- Cada Clase tiene asociado un archivo de código fuente (Source Code)
 - **Circulo.java**
- Al compilar, se genera un archivo de código binario
 - **Circulo.class**
 - Si existen errores, este archivo no se genera
- Este archivo es interpretado y ejecutado por la Máquina Virtual de Java
 - Existen intérpretes para cada plataforma.
- Las clases tiene una sintaxis particular para definir los métodos, atributos, variables, etc.
- A través del código, podemos crear instancias de objetos de otras clases programáticamente, e invocar sus métodos.



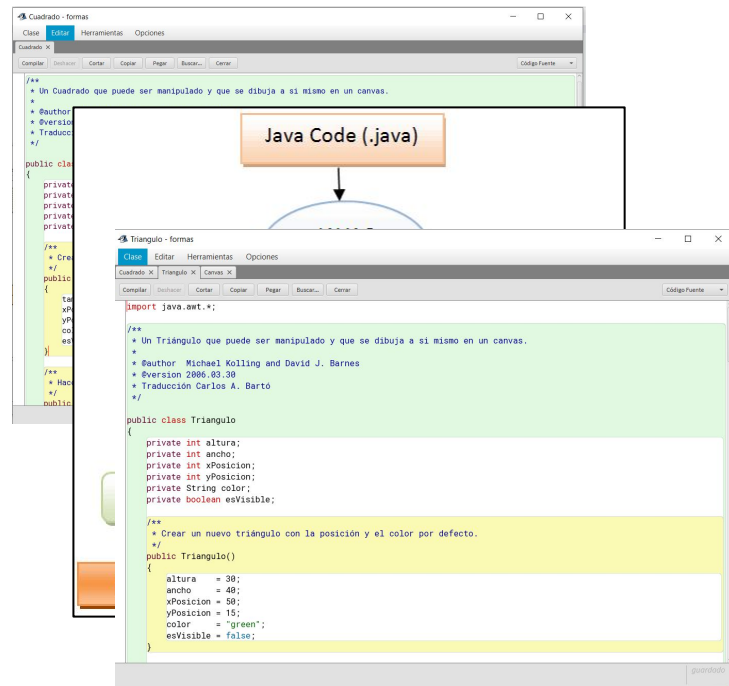
BlueJ - Código Fuente

- Cada Clase tiene asociado un archivo de código fuente (Source Code)
 - **Circulo.java**
- Al compilar, se genera un archivo de código binario
 - **Circulo.class**
 - Si existen errores, este archivo no se genera
- Este archivo es interpretado y ejecutado por la Máquina Virtual de Java
 - Existen intérpretes para cada plataforma.
- Las clases tienen una sintaxis particular para definir los métodos, atributos, variables, etc.
- A través del código, podemos crear instancias de objetos de otras clases programáticamente, e invocar sus métodos.



BlueJ - Código Fuente

- Cada Clase tiene asociado un archivo de código fuente (Source Code)
 - **Circulo.java**
- Al compilar, se genera un archivo de código binario
 - **Circulo.class**
 - Si existen errores, este archivo no se genera
- Este archivo es interpretado y ejecutado por la Máquina Virtual de Java
 - Existen intérpretes para cada plataforma.
- Las clases tienen una sintaxis particular para definir los métodos, atributos, variables, etc.
- A través del código, podemos crear instancias de objetos de otras clases programáticamente, e invocar sus métodos.

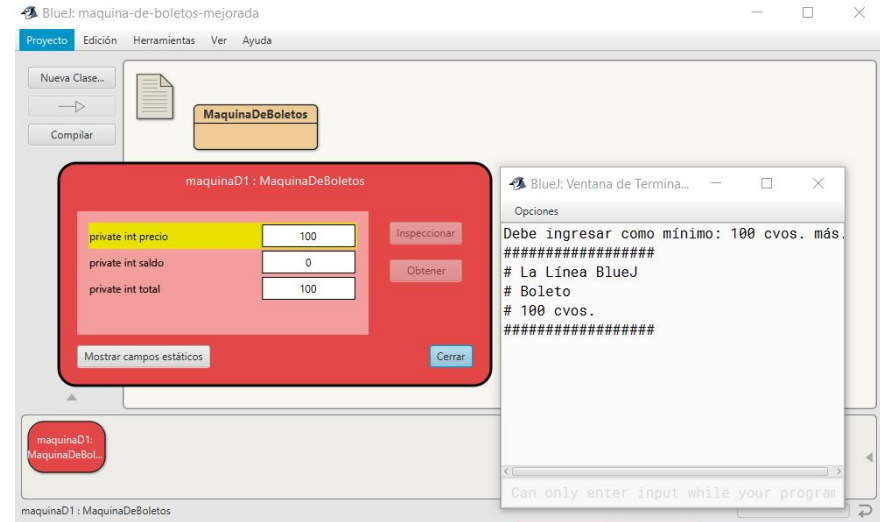


[illegible]

Definición de Clases

Proyecto “Maquina de Boletos Simple”

- Abrir el proyecto
 - Compilar
- Crear una instancia
 - Asignar precio
- Interactuar con el Objeto
 - Revisar el Estado
 - Usar los métodos
 - Imprimir boleto
 - Agregar dinero
- Abrir el código fuente de la Clase
 - Explorar la clase
 - Tratar de Entender la Sintaxis
 - Interpretar la Estructura



Estructura de una Clase

```
public class Circulo
```

```
{
```

//Atributos o campos

```
private int diametro;
```

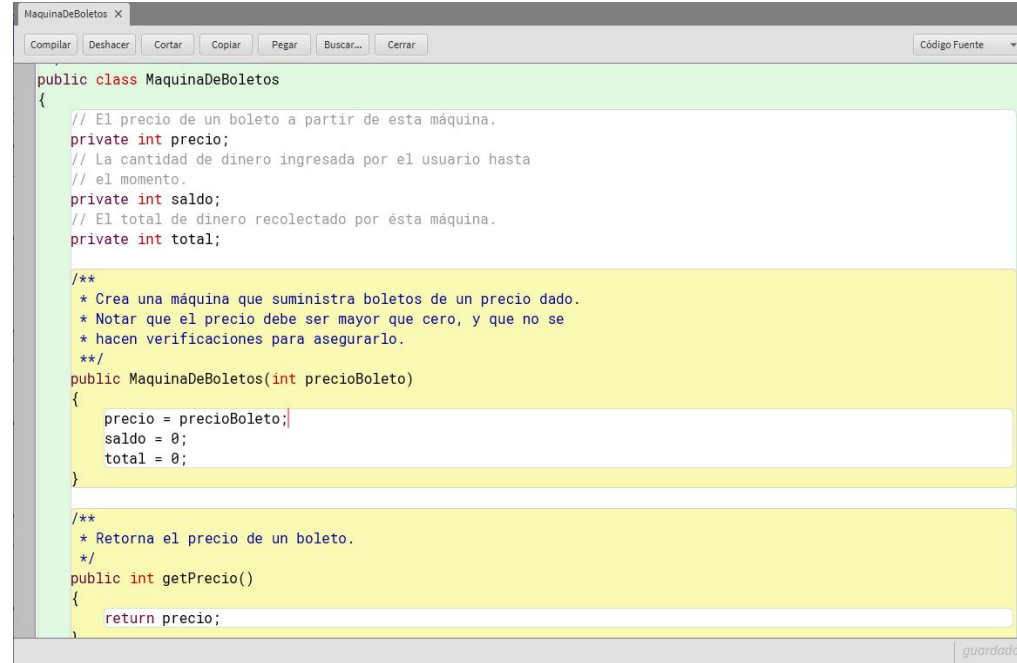
//Constructores

```
public Circulo () { }
```

//Metodos

```
public int getDiametro() { }
```

```
}
```



The screenshot shows a code editor window titled 'MaquinaDeBoletos X'. The editor contains the following Java code:

```
public class MaquinaDeBoletos
{
    // El precio de un boleto a partir de esta máquina.
    private int precio;
    // La cantidad de dinero ingresada por el usuario hasta
    // el momento.
    private int saldo;
    // El total de dinero recolectado por ésta máquina.
    private int total;

    /**
     * Crea una máquina que suministra boletos de un precio dado.
     * Notar que el precio debe ser mayor que cero, y que no se
     * hacen verificaciones para asegurarlo.
     */
    public MaquinaDeBoletos(int precioBoleto)
    {
        precio = precioBoleto;
        saldo = 0;
        total = 0;
    }

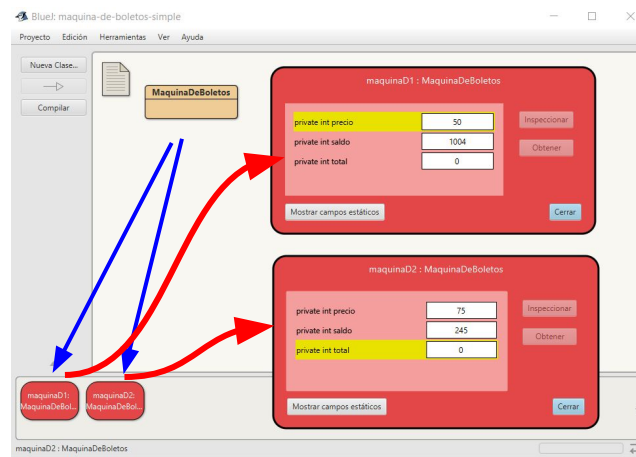
    /**
     * Retorna el precio de un boleto.
     */
    public int getPrecio()
    {
        return precio;
    }
}
```

The editor interface includes a menu bar with 'Compilar', 'Deshacer', 'Cortar', 'Copiar', 'Pegar', 'Buscar...', and 'Cerrar'. A 'Código Fuente' dropdown is visible in the top right corner. The bottom right corner of the editor shows a 'guardado' status.

Atributos - Campos - Variable de Instancia

- Los campos o atributos almacenan los valores, y tienen un **identificador** (por convención en **minúsculas**)
- Pueden ser de un **tipo** en particular (**int**, **boolean**, **double**, **float**, etc)
- Tienen una nivel de **acceso** (**public**, **private**, **protected**)
- Cada instancia de una clase (i.e., cada objeto) tiene su propio valor
- Por eso se conocen como variables de instancia
- El estado de cada objeto está dado por sus atributos

```
public class MaquinaDeBoletos
{
    // El precio de un boleto a partir de esta máquina.
    private int precio;
    // La cantidad de dinero ingresada por el usuario hasta
    // el momento.
    private int saldo;
    // El total de dinero recolectado por ésta máquina.
    private int total;
}
```



Métodos

- Los métodos implementan las acciones que el objeto puede realizar.
- Poseen un **nombre** o **identificador** (por convención, en **minúscula**)
- Tienen una nivel de **acceso** (**public**, **private**, **protected**)
- Definen un valor de **retorno** (algún **tipo** de dato o **void** si no devuelven ningún valor)
- Declaran una lista de **parámetros** (que puede ser vacía si no requiere)
- La primer línea, que declara todo esto es la “firma” o “signature” del método
- El cuerpo del método implementa la funcionalidad

```
/**
 * Imprime un boleto.
 * Actualiza el total de dinero recolectado y
 * pone el saldo en cero.
 */
public void imprimirBoleto()
{
    // Simulata la impresión de un boleto.
    System.out.println("#####");
    System.out.println("# La Línea BlueJ");
    System.out.println("# Boleto");
    System.out.println("# " + precio + " cvos.");
    System.out.println("#####");
    System.out.println();

    // Actualizar el total recolectado con el saldo.
    total = total + saldo;
    // Limpiar el saldo.
    saldo = 0;
}
```

Constructores

- Los constructores son **métodos** que se ejecutan al “crear” o “instanciar” objetos de una clase.
- Sirven para asignar un valor inicial a las variables de instancia.
- Una clase puede tener más de un constructor.
 - Pueden tener listas de parámetros diferentes
 - Al crear un nuevo objeto, hay que elegir qué constructor usar (según los parámetros que se pasen)
- A diferencia de los métodos normales, los constructores
 - Se llaman igual que la clase (mayúscula)
 - No deben declarar ningún valor de retorno (ni siquiera void)
- Si no se define ningún constructor, el compilador crea uno vacío por defecto.

```
/**
 * Crea una máquina que suministra boletos de un precio dado
 * Notar que el precio debe ser mayor que cero, y que no se
 * hacen verificaciones para asegurarlo.
 */
public MaquinaDeBoletos(int precioBoleto)
{
    precio = precioBoleto;
    saldo = 0;
    total = 0;
}
```

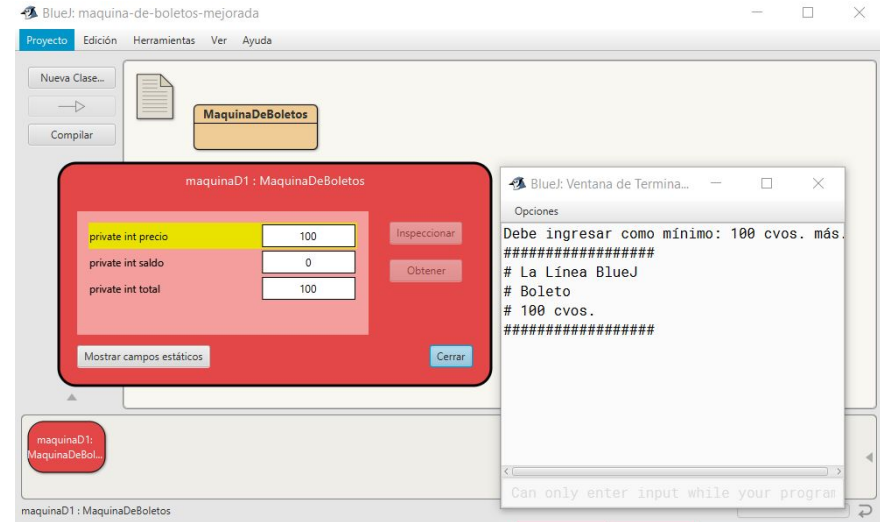
Getters & Setters

- Los getters son métodos usados para proveer información de las variables de un objeto.
- Los getters devuelven un **tipo** de dato (a través de **return**), y no requieren parámetros
- Por convención usamos como nombre el nombre del campo que queremos acceder, precedido de “get”
- Los setters son métodos usados para establecer valores a las variables de un objeto.
- Los setters no devuelven ningún valor (**void**), y no toman como parámetro el valor a setear.
- Por convención usamos como nombre el nombre del campo que queremos setear, precedido de “set”

```
/**
 * Retorna el precio de un boleto.
 */
public int getPrecio()
{
    re /**
     * Establece un nuevo precio de un boleto.
     */
     public void setPrecio(int nuevoPrecio)
     {
         precio = nuevoPrecio;
     }
}
```

Proyecto “Maquina de Boletos Mejorada”

- Analizar comportamiento de “Máquina de Boletos Simple”
 - No verifica montos ingresados
 - No da vuelto
 - No se inicializa correctamente
- Propuestas de mejora
 - Validar ingreso de dinero
 - Expedir boletos basado en saldo
 - Reintegrar vuelto



Variables y Asignación de valores

- Las variables **almacenan** valores
- Se declaran asignándoles un **tipo de dato** y un **identificador** (nombre, siempre en **minúscula**)
- La operación de asignación utiliza el **operador =**
- Existe **precedencia**
 - Primero se resuelve la expresión a la derecha
 - Finalmente se asigna el resultado a la variable
- Las **variables de “instancia”** o campos
 - **Mantienen su valor durante la existencia del objeto**
- Se pueden declarar **variables locales**
 - No necesitan modificador de visibilidad
 - Su existencia es dinámica,
 - **Desaparecen de memoria al finalizar la ejecución del scope donde fueron declaradas**



Sentencias de Ejecución Condicional (if-else)

```
// Asumimos que "y" es un int
if (y > 10) {
    // Este bloque de código se ejecuta si y>10
    System.out.println ("Numero mayor");
} else {
    // Este bloque de código se ejecuta en el caso contrario
    System.out.println ("Numero mayor");
}
// El flujo del programa sigue normalmente
```

A word cloud of programming concepts in Spanish, arranged in a circular pattern. The words are in various shades of green and purple. The most prominent words are 'Constructor', 'Metodo', 'Atributo', 'Campo', 'Variable local', 'Parametros', 'If-Else', 'Asignacion', 'Getter', and 'Setter'. Other visible words include 'Tipo de Datos', 'Constructor', 'Metodo', 'Atributo', 'Campo', 'Variable local', 'Parametros', 'If-Else', 'Asignacion', 'Getter', and 'Setter'.